

Lab0：实验环境搭建

本章完成的工作

本章从零开始搭建了 uCore-Tutorial 实验环境，主要完成以下任务：

1. 安装 Ubuntu 24.04 LTS 作为开发环境
2. 部署 RISC-V 交叉编译工具链（GCC 10.1.0 + musl 10.2.1）
3. 从源码编译安装 QEMU 7.0.0 模拟器
4. 配置 GDB 调试环境
5. 验证环境配置的正确性，成功运行 uCore ch1 分支

一、系统环境配置

1.1 操作系统选择

本实验选择 Ubuntu 24.04 LTS 作为开发环境。相比清华原版推荐的 Ubuntu 18.04/20.04，这是更新的 LTS 版本，经验证与实验完全兼容。

安装方式：使用 VMware 安装 Ubuntu 24.04 虚拟机

WSL2 也是可行的选择，安装命令如下（供参考）：

```
# 以管理员身份打开 PowerShell
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
wsl --set-default-version 2
wsl --install -d Ubuntu-24.04
```

1.2 基础环境更新

进入系统后，首先更新软件包：

```
sudo apt update
sudo apt upgrade -y
```

安装基础开发工具：

```
sudo apt install -y build-essential git wget curl
```

二、RISC-V 交叉编译工具链安装

2.1 riscv64-unknown-elf-gcc 安装

下载 SiFive 预编译的 RISC-V 工具链：

```
cd /usr/local

# 下载工具链
sudo wget https://static.dev.sifive.com/dev-tools/freedom-tools/v2020.08/riscv64-unknown-elf-gcc-10.1.0-2020.08.2-x86_64-linux-ubuntu14.tar.gz

# 解压并重命名
sudo tar xzf riscv64-unknown-elf-gcc-10.1.0-2020.08.2-x86_64-linux-ubuntu14.tar.gz
sudo mv riscv64-unknown-elf-gcc-10.1.0-2020.08.2-x86_64-linux-ubuntu14 riscv64-unknown-elf-gcc

# 清理压缩包
sudo rm riscv64-unknown-elf-gcc-10.1.0-2020.08.2-x86_64-linux-ubuntu14.tar.gz
```

2.2 riscv64-linux-musl-gcc 安装

musl-gcc 用于编译用户态程序：

```
cd /usr/local

# 从清华网盘下载 musl 工具链
sudo wget -O riscv64-linux-musl-cross.tgz
"https://cloud.tsinghua.edu.cn/f/fb3c598e7e214a828e6b/?dl=1"

# 解压并清理
sudo tar xzf riscv64-linux-musl-cross.tgz
sudo rm riscv64-linux-musl-cross.tgz
```

2.3 环境变量配置

编辑 `~/.bashrc`，在文件末尾添加：

```
# RISC-V 工具链
export PATH="/usr/local/riscv64-unknown-elf-gcc/bin:$PATH"
export PATH="/usr/local/riscv64-linux-musl-cross/bin:$PATH"
```

使配置生效：

```
source ~/.bashrc
```

2.4 安装验证

```
riscv64-unknown-elf-gcc --version  
riscv64-linux-musl-gcc --version
```

输出结果:

```
riscv64-unknown-elf-gcc (SiFive GCC 10.1.0-2020.08.2) 10.1.0  
riscv64-linux-musl-gcc (GCC) 10.2.1 20210227
```

三、QEMU 模拟器编译安装

3.1 依赖包安装

编译 QEMU 需要以下依赖:

```
sudo apt install -y autoconf automake autotools-dev curl libmpc-dev libmpfr-dev libgmp-dev \  
gawk build-essential bison flex texinfo gperf libtool patchutils bc ninja-build \  
zlib1g-dev libexpat-dev pkg-config libglib2.0-dev libpixman-1-dev git tmux python3
```

3.2 源码编译

```
cd ~  
  
# 从清华网盘下载源码  
wget -O qemu-7.0.0.tar.xz "https://cloud.tsinghua.edu.cn/f/8ba524dbede24ce79d06/?dl=1"  
  
# 解压  
tar xJf qemu-7.0.0.tar.xz  
  
# 编译 (约需 5-10 分钟)  
cd qemu-7.0.0  
./configure --target-list=riscv64-softmmu,riscv64-linux-user  
make -j$(nproc)
```

3.3 环境变量配置

编辑 `~/.bashrc`, 添加:

```
# QEMU  
export PATH="$HOME/qemu-7.0.0/build:$PATH"  
export PATH="$HOME/qemu-7.0.0/build/riscv64-softmmu:$PATH"  
export PATH="$HOME/qemu-7.0.0/build/riscv64-linux-user:$PATH"
```

使配置生效:

```
source ~/.bashrc
```

3.4 安装验证

```
qemu-system-riscv64 --version
```

输出结果:

```
QEMU emulator version 7.0.0
```

四、GDB 调试环境

GDB 调试器已包含在 riscv64-unknown-elf-gcc 工具链中，无需额外安装。

验证:

```
riscv64-unknown-elf-gdb --version
```

输出结果:

```
GNU gdb (SiFive GDB 9.1.0-2020.08.2) 9.1
```

五、其他工具

安装 cmake:

```
sudo apt install -y cmake
```

验证:

```
cmake --version  
make --version
```

输出结果:

```
cmake version 3.28.3  
GNU Make 4.3
```

六、实验代码获取

6.1 代码仓库说明

清华 uCore 实验包含两个仓库:


```
[rustsbi] Implementation      : RustSBI-QEMU Version 0.2.0-alpha.2
[rustsbi] Platform Name      : riscv-virtio,qemu
[rustsbi] Platform SMP       : 1
[rustsbi] Platform Memory    : 0x80000000..0x88000000
[rustsbi] Boot HART          : 0
[rustsbi] Device Tree Region : 0x87000000..0x87000ef2
[rustsbi] Firmware Address   : 0x80000000
[rustsbi] Supervisor Address : 0x80200000
[rustsbi] pmp01: 0x00000000..0x80000000 (-wr)
[rustsbi] pmp02: 0x80000000..0x80200000 (---)
[rustsbi] pmp03: 0x80200000..0x88000000 (xwr)
[rustsbi] pmp04: 0x88000000..0x00000000 (-wr)

hello world!
[ERROR 0]stext: 0x0000000080200000
[WARN 0]etext: 0x0000000080201000
[INFO 0]sroda: 0x0000000080201000
[DEBUG 0]eroda: 0x0000000080202000
[DEBUG 0]sdata: 0x0000000080202000
[INFO 0]edata: 0x0000000080202000
[WARN 0]sbss : 0x0000000080212000
[ERROR 0]ebss : 0x0000000080212000
[PANIC 0] os/main.c:39: ALL DONE
```

环境验证成功。

退出 QEMU：按 `Ctrl+A`，再按 `x`

七、环境版本汇总

组件	版本	说明
操作系统	Ubuntu 24.04.3 LTS	虚拟机
RISC-V GCC	SiFive GCC 10.1.0-2020.08.2	内核编译器
musl-gcc	GCC 10.2.1	用户态编译器
QEMU	7.0.0	RISC-V 模拟器
GDB	SiFive GDB 9.1.0	调试器
cmake	3.28.3	构建工具
make	GNU Make 4.3	构建工具

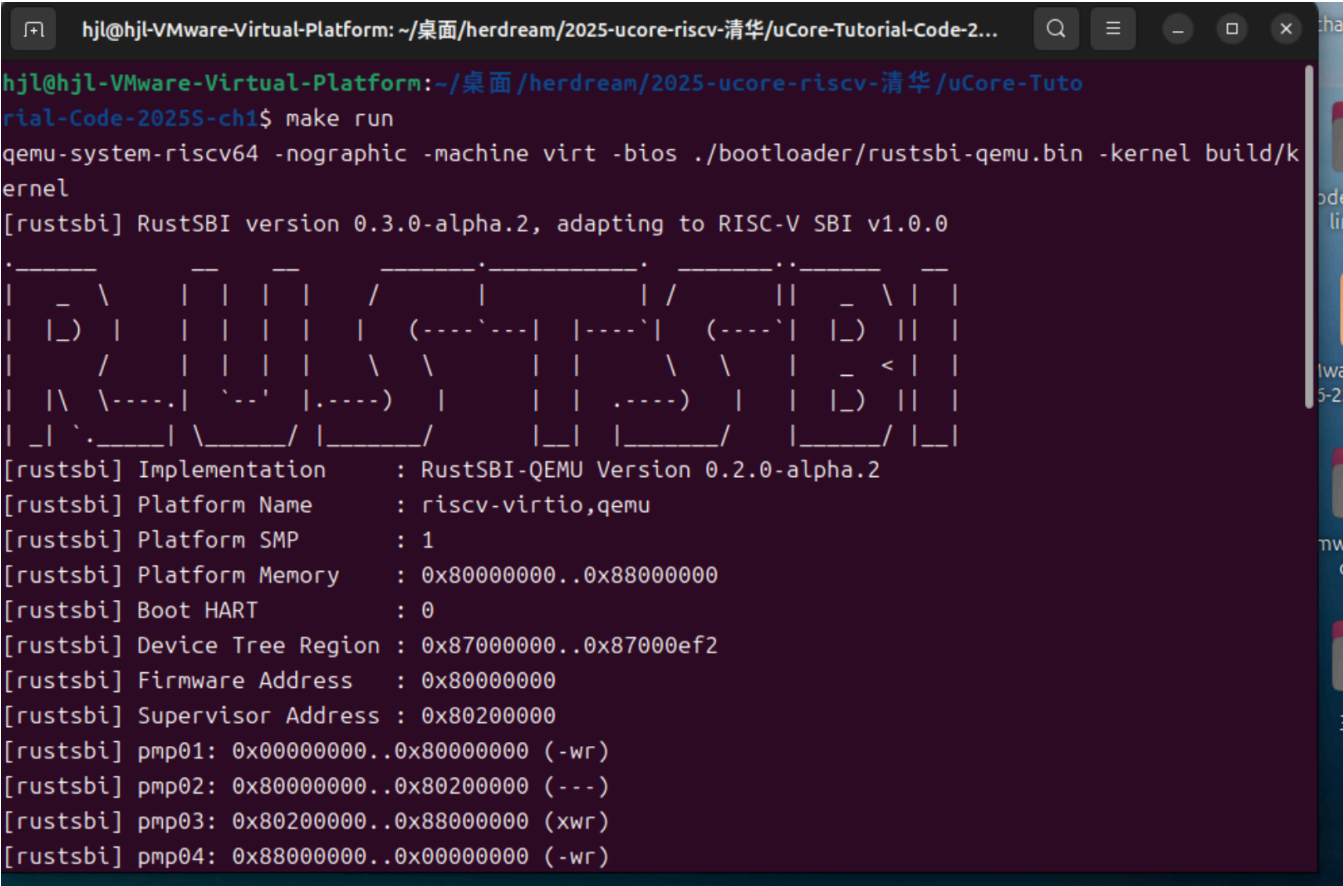
八、与清华原版的区别

项目	清华原版	本实验配置	说明
操作系统	Ubuntu 18.04/20.04	Ubuntu 24.04	更新的 LTS 版本，兼容性良好

项目	清华原版	本实验配置	说明
GDB	8.3.0	9.1.0	工具链自带的更新版本

其他组件（GCC 10.1.0、musl-gcc 10.2.1、QEMU 7.0.0）与清华原版完全一致。

九、验证截图



十、遇到的问题及解决

暂未遇到问题

实验总结

本章成功搭建了 uCore 实验环境，完成了以下工作：

- ✓ Ubuntu 24.04 环境配置
- ✓ RISC-V GCC 工具链安装
- ✓ musl-gcc 工具链安装
- ✓ QEMU 7.0.0 源码编译安装
- ✓ GDB 调试环境配置
- ✓ 环境验证通过

所有组件版本符合实验要求，可以进行后续实验。